

Topic 2A: What Next-Token Prediction Learns

Last updated: September 15, 2026

Surbhi Goel

Acknowledgements: These notes draw on Cover and Thomas [1] for likelihood and KL divergence, Alvarez-Melis, Garg, and Kalai [2] for likelihood and distinguishability, Cao and Vempala [3] for computational aspects of next-token prediction, and Rohatgi, Block, Huang, Krishnamurthy, and Foster [4] for the misspecification example.

AI Disclosure: These notes were drafted and revised with the assistance of AI tools. I have reviewed the content and take responsibility for it. If you find an error, please let me know.

In previous lectures, we studied what a learner can generate from valid examples and why a model that tries to reproduce their frequencies may still hallucinate. Here we study the objective used to fit those frequencies. *What does learning to predict the next token tell us about the probabilities of complete outputs?*

Recall the pretraining procedure from previous lectures. At every position in a document, the preceding text is the input and the token that follows is the target. Log loss rewards assigning more probability to the observed token. The same loss is used whether the token completes a fact, closes a parenthesis, or follows a step in an argument. Yet the properties we care about often concern the complete document: whether it mentions a particular course, keeps distant parts of the text consistent, or makes a false claim.

1 Next-Token Prediction and Document Likelihood

Write p for the source distribution and q for the model. In the previous lecture, each document was represented by its factoid. Here we keep the sequence of tokens. Let $\mathcal{V}$ be a finite vocabulary and suppose every document has length T , so the universe of documents is $\mathcal{U} = \mathcal{V}^T$.¹

Predicting the next token. Take one document $x_{1:T} = (x_1, \dots, x_T)$. It provides a prediction problem at every position. To predict token t , the model receives the preceding tokens, which we call the *prefix* and write as $x_{<t} = (x_1, \dots, x_{t-1})$. At the first position, the prefix is empty.

Given this input, the model returns a distribution $q(\cdot \mid x_{<t})$ over the vocabulary. For each possible next token $v \in \mathcal{V}$, it assigns probability $q(v \mid x_{<t})$. The token x_t that actually follows in the document is the prediction target.

At position $t + 1$, the input is $(x_1, \dots, x_t)$. This prefix still comes entirely from the document, even if the model assigned the most probability to a different token at position t . Supplying the observed prefix at each position is called *teacher forcing*.

¹Bounded-length documents can be represented by adding an end-of-sequence symbol and padding to length T .

Scoring a prediction. The score at position t uses the probability assigned to the observed token x_t . Its *log loss* is

$$-\log q(x_t \mid x_{<t}).$$

The loss is zero when this probability is one, and grows as the probability decreases. Increasing the probability of the observed token therefore improves the score, even if another token still has the highest probability. All logarithms are natural ($\log = \ln$), and we set $-\log 0 = +\infty$.

From token losses to document likelihood. Adding these losses over the positions gives the loss on one document:

$$L_q(x_{1:T}) := \sum_{t=1}^T -\log q(x_t \mid x_{<t}). \quad (1)$$

What does this sum measure? The probability chain rule gives

$$q(x_{1:T}) = \prod_{t=1}^T q(x_t \mid x_{<t}). \quad (2)$$

Taking the negative logarithm turns this product into exactly the sum above:

$$L_q(x_{1:T}) = -\log q(x_{1:T}).$$

Thus the sum of next-token log losses is the *negative log-likelihood* of the complete document. Reducing this sum increases the probability assigned to that document.

For a corpus of n documents $x^{(1)}, \dots, x^{(n)}$, averaging gives the *empirical loss*

$$\widehat{L}_{\text{seq}}(q) = \frac{1}{n} \sum_{i=1}^n L_q(x^{(i)}) = -\frac{1}{n} \log \prod_{i=1}^n q(x^{(i)}).$$

If documents are modeled as independent draws from q , the product above is the probability of the observed corpus. With the corpus fixed, this product as a function of q is the *likelihood*. *Maximum likelihood estimation* (MLE) chooses the model in a given class that maximizes this likelihood. The identity above shows that this is equivalent to minimizing empirical next-token log loss. Next-token training therefore optimizes the maximum-likelihood objective for complete documents.

Generation. After training, the model must supply its own prefixes. Write $Y_{1:T}$ for the generated document. We sample

$$Y_1 \sim q(\cdot), \quad Y_t \sim q(\cdot \mid Y_{<t}), \quad t = 2, \dots, T,$$

so at each step the model conditions on its own earlier choices² (Figure 1).

²Greedy decoding, temperature changes, and truncation rules such as top- k produce a different output distribution. Our guarantees concern sampling from q .

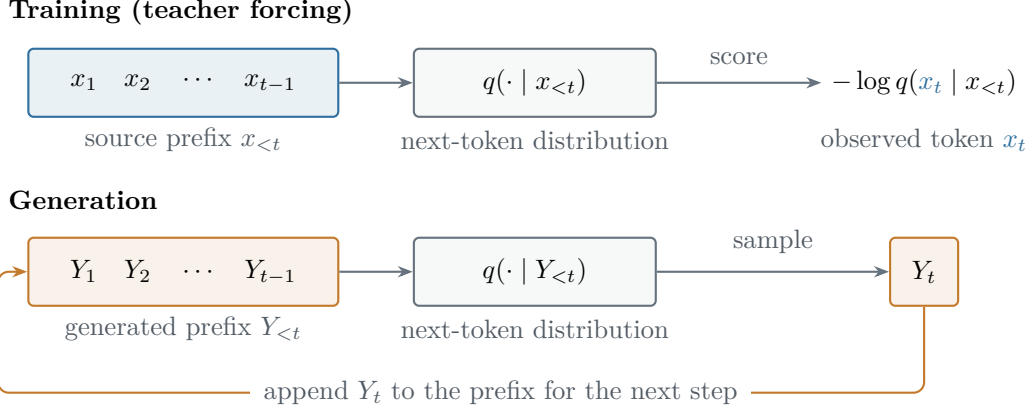


Figure 1: The model receives the whole prefix in both cases. Training scores its prediction on the next source token, and the next input still comes from the source. Generation samples a token and adds it to the prefix, so each choice changes the input for the following step.

By (2), this sampling procedure generates documents with distribution q . The same document probability therefore determines both the training score and how often the model generates that document.

2 The Source Distribution Minimizes Population Loss

The empirical loss averages over the training corpus. Averaging the same score over a fresh document $X \sim p$ gives the *population loss*,

$$L_{\text{seq}}(q) = \mathbb{E}_{X \sim p}[-\log q(X)]. \quad (3)$$

Every document the source can produce contributes to this expectation, whether or not it appeared in the corpus. For now, q may be any distribution.

2.1 Entropy and Excess Loss

Even a model that knows the source distribution may not know which token comes next. For English text, after *She opened the*, the next word might be *door*, *window*, or *book*. A model that knows how often each continuation occurs must still divide its probability among them, and so incurs positive log loss on the continuation that appears.

Consider a model that uses the source probabilities, so $q = p$. Its expected loss is the *entropy* of the source,

$$H(p) = \mathbb{E}_{X \sim p}[-\log p(X)] = - \sum_x p(x) \log p(x),$$

where $0 \log 0 = 0$. It measures uncertainty about which document will be drawn from p . On a document x that the source can produce, the extra loss from using q instead of p is

$$-\log q(x) - [-\log p(x)] = \log \frac{p(x)}{q(x)}.$$

Assigning half the source's probability to x costs an additional $\log 2$ when that document appears. Assigning more probability than the source does gives a negative difference on that document.

Averaging these differences under p gives the *Kullback–Leibler (KL) divergence*, also called relative entropy:

$$\text{KL}(p\|q) := \mathbb{E}_{X \sim p} \left[\log \frac{p(X)}{q(X)} \right] = \sum_{x:p(x)>0} p(x) \log \frac{p(x)}{q(x)}. \quad (4)$$

A summand is $+\infty$ when $p(x) > 0$ and $q(x) = 0$. Thus the population loss separates into

$$L_{\text{seq}}(q) = H(p) + \text{KL}(p\|q).$$

The KL term measures the model’s *excess loss* beyond the entropy of the source. The order of KL’s arguments matters: p supplies the documents, while q supplies the probabilities used to score them. Reversing these roles generally changes the value.

Could a model do better than the source by concentrating more probability on the most common documents? This would reduce the loss whenever one of those documents appears. But the less common documents would still occur at their original frequencies under p , and they would now receive less probability and incur larger losses. Normalization prevents this trade from reducing average loss. Gibbs’ inequality formalizes this observation; see Cover and Thomas [1].

Proposition 2.1 (Population minimum of log loss). *For every distribution q on $\mathcal{V}^T$,*

$$L_{\text{seq}}(q) \geq H(p),$$

with equality if and only if $q = p$.

Proof. It suffices to show that $\text{KL}(p\|q) \geq 0$, with equality only at $q = p$. Let $S = \{x : p(x) > 0\}$ be the support of the source. If $q(x) = 0$ for some $x \in S$, its expected loss is infinite and the claim follows. Otherwise, all ratios $q(x)/p(x)$ on S are positive, and

$$\begin{aligned} \text{KL}(p\|q) &= \sum_{x \in S} p(x) \left[-\log \frac{q(x)}{p(x)} \right] \\ &\geq \sum_{x \in S} p(x) \left[1 - \frac{q(x)}{p(x)} \right] = 1 - \sum_{x \in S} q(x) \geq 0. \end{aligned}$$

The first inequality uses $-\log z \geq 1 - z$, with equality only at $z = 1$. The last inequality uses $\sum_{x \in S} q(x) \leq 1$.

For equality, every ratio $q(x)/p(x)$ on S must equal one. These probabilities already sum to one, leaving q no probability outside S . Hence $q = p$. Conversely, choosing $q = p$ gives loss $H(p)$. $\square$

2.2 What Next-Token Loss Asks of Each Prefix

Proposition 2.1 characterizes the optimum in terms of complete documents. Suppose $\text{KL}(p\|q) < \infty$. Applying the probability chain rule to p and q connects this sequence-level optimum to individual tokens:

$$\log \frac{p(X_{1:T})}{q(X_{1:T})} = \sum_{t=1}^T \log \frac{p(X_t \mid X_{<t})}{q(X_t \mid X_{<t})}.$$

For a fixed prefix, averaging its log ratio over the next source token gives a conditional KL divergence. Averaging in turn over source prefixes yields the chain rule for relative entropy [1]:

$$\text{KL}(p\|q) = \sum_{t=1}^T \mathbb{E}_{X_{<t} \sim p} \text{KL}(p(\cdot \mid X_{<t})\|q(\cdot \mid X_{<t})). \quad (5)$$

Equation (5) identifies excess document loss with the sum of conditional next-token divergences.

Because every term in (5) is nonnegative, errors cannot cancel across prefixes. Reaching the minimum requires the model to match the source’s next-token distribution at *every* prefix with positive source probability. A predictor that treats two such prefixes identically when their source conditionals differ must incur positive excess loss.

In (5), prefixes are drawn from the source p , not from the model q . At $q = p$, source and generated prefixes have the same distribution. Away from the optimum, their distributions may differ, and the model may reach prefixes that the source rarely or never produces.

Predictions after a supplied prompt. In an application, we often choose the prompt ourselves. Let s be a prefix of length $t - 1 < T$, with source probability $p(s) = \Pr_{X \sim p}[X_{<t} = s] > 0$. Its contribution to (5) is its next-token divergence multiplied by $p(s)$. Since all the other contributions are nonnegative,

$$\text{KL}(p(\cdot | s) \| q(\cdot | s)) \leq \frac{\text{KL}(p \| q)}{p(s)}.$$

Thus a model can have small average excess loss while predicting poorly after a rare prompt. Grouping the chain rule at the end of the prompt gives the same inequality for the distributions of complete continuations after s .

3 Likelihood and Distinguishability

Population loss averages over source documents. We would like to use it to understand documents generated by the model: how often will they have a property we care about?

Let $B \subseteq \mathcal{U}$ be a property we can check on a completed document, such as having mismatched opening and closing tags. Its probability is $p(B)$ under the source and $q(B)$ under the model. The gap $|q(B) - p(B)|$ measures how accurately the model reproduces its frequency. If a property occurs more often in generated text, observing it is evidence that a document came from the model. A test that checks this property therefore gives us a way to measure the frequency error.

3.1 Property Frequencies and Distinguishing Tests

Suppose we are given a document drawn from p or q , each with probability $1/2$, without being told which. A guess that ignores the document is correct with probability $1/2$.

Suppose $q(B) > p(B)$. Since B occurs more often in model documents, guess *model* when B occurs and *source* otherwise.

On a source document, this rule is correct when B does not occur, with probability $1 - p(B)$. On a model document, it is correct when B occurs, with probability $q(B)$. Averaging the two cases gives the overall accuracy

$$\frac{1}{2}(1 - p(B)) + \frac{1}{2}q(B) = \frac{1}{2} + \frac{q(B) - p(B)}{2}. \quad (6)$$

If the discrepancy has the other sign, reverse the guesses. The absolute frequency gap $|q(B) - p(B)|$ is called the test’s *distinguishing advantage*; its edge over random guessing is half that gap.

Taking the largest gap over all choices of B gives the *total variation distance*:

$$\text{TV}(p, q) := \max_{B \subseteq \mathcal{U}} |p(B) - q(B)| = \frac{1}{2} \sum_x |p(x) - q(x)|.$$

The maximizing event can be taken as $\{x : q(x) > p(x)\}$: its excess mass equals the total deficit on the complement. The best distinguishing accuracy is therefore $1/2 + \text{TV}(p, q)/2$. Small TV distance means that every property has nearly the same frequency under p and q .

We already know that $\text{KL}(p\|q)$ is the excess population log loss. Pinsker’s inequality [1] turns this loss into a bound on property frequencies:

$$\underbrace{\max_{B \subseteq \mathcal{U}} |p(B) - q(B)|}_{\text{largest frequency error}} = \text{TV}(p, q) \leq \sqrt{\frac{\text{KL}(p\|q)}{2}} = \sqrt{\frac{L_{\text{seq}}(q) - H(p)}{2}}. \quad (7)$$

Excess document loss at most $2\varepsilon^2$ therefore guarantees that *every* property’s frequency is correct to within ε . If the source never produces mismatched tags, for example, the model’s mismatch probability is at most ε . This extends the population-optimum result: small excess loss gives approximate agreement of all property frequencies, and zero excess loss gives exact agreement.

3.2 Correcting a Frequency Mismatch

Pinsker’s inequality controls every property at once when excess loss is small. A restricted model class may have large minimum KL divergence, even if it can reproduce some properties exactly. For a single property, we can ask more directly: if its frequency is wrong, does correcting it lower loss?

Suppose $q(B) > p(B)$, so the model puts too much probability on B . Multiply the probability of each document in B by $e^{-\eta}$, where $\eta > 0$ is a step size. The total remaining mass is $1 - q(B) + q(B)e^{-\eta}$. Dividing by this mass gives the corrected distribution

$$q^{(\eta)}(x) = \frac{q(x)e^{-\eta \mathbf{1}\{x \in B\}}}{1 - q(B) + q(B)e^{-\eta}}. \quad (8)$$

This changes the total mass assigned to B while preserving relative probabilities within B and within B^c . The same formula gives q at $\eta = 0$ and upweights B when $\eta < 0$.

The following proposition is a population, binary-test version of the likelihood-boosting lemma of Alvarez-Melis, Garg, and Kalai [2, Lemma 1].³

Proposition 3.1 (Quantitative loss improvement). *Assume $L_{\text{seq}}(q) < \infty$, equivalently $q(x) > 0$ whenever $p(x) > 0$, and suppose $q(B) > p(B)$. For every step size $\eta \geq 0$, the correction in (8) satisfies*

$$L_{\text{seq}}(q) - L_{\text{seq}}(q^{(\eta)}) \geq \eta(q(B) - p(B)) - \frac{\eta^2}{2}.$$

In particular, choosing $\eta = q(B) - p(B)$ gives a decrease of at least $\eta^2/2$.

Proof. By the definition of $q^{(\eta)}$,

$$\begin{aligned} L_{\text{seq}}(q) - L_{\text{seq}}(q^{(\eta)}) &= \mathbb{E}_{X \sim p} \left[\log \frac{q^{(\eta)}(X)}{q(X)} \right] \\ &= -\eta p(B) - \log(1 - q(B) + q(B)e^{-\eta}). \end{aligned} \quad (9)$$

Downweighting B costs $\eta p(B)$ in expected log probability; normalization increases every log probability by the second term. To bound this gain, apply $-\log(1-u) \geq u$ with $u = q(B)(1-e^{-\eta}) \in [0, 1]$:

$$-\log(1 - q(B) + q(B)e^{-\eta}) \geq q(B)(1 - e^{-\eta}).$$

³Their lemma uses empirical frequencies and also allows tests with bounded real-valued scores.

Using $1 - e^{-\eta} \geq \eta - \eta^2/2$ for $\eta \geq 0$ in (9), we obtain

$$\begin{aligned} L_{\text{seq}}(q) - L_{\text{seq}}(q^{(\eta)}) &\geq -\eta p(B) + q(B) \left(\eta - \frac{\eta^2}{2} \right) \\ &= \eta(q(B) - p(B)) - \frac{\eta^2 q(B)}{2} \\ &\geq \eta(q(B) - p(B)) - \frac{\eta^2}{2}, \end{aligned}$$

where the last inequality uses $q(B) \leq 1$. Taking $\eta = q(B) - p(B)$ gives the claimed $\eta^2/2$ decrease. $\square$

The frequency error contributes the positive term $\eta(q(B) - p(B))$, while the penalty for the step is quadratic in η . Thus every $0 < \eta < 2(q(B) - p(B))$ gives a strict improvement. If $q(B) < p(B)$, applying the argument to B^c gives the correction in (8) with a negative step size.

3.3 Matching Properties in a Restricted Class

Let $B_1, \dots, B_m$ be properties of documents. We say that $\mathcal{Q}$ is *closed under reweighting these properties* if, for every model $q \in \mathcal{Q}$, every $i \in \{1, \dots, m\}$, and every $\eta \in \mathbb{R}$, the corrected distribution in (8) with $B = B_i$ also belongs to $\mathcal{Q}$. The condition applies to every model obtained after earlier corrections, so any finite sequence of these reweightings stays in the class.

Under this assumption, any population-loss minimizer q^* with finite loss matches all the property frequencies:

$$q^*(B_i) = p(B_i) \quad \text{for every } i \in \{1, \dots, m\},$$

even if $p \notin \mathcal{Q}$.

To see why, fix i . If $q^*(B_i) > p(B_i)$, Proposition 3.1 gives a correction with strictly smaller loss. If $q^*(B_i) < p(B_i)$, apply the argument to B_i^c ; this is the correction for B_i with a negative step size. Closure puts either corrected model in $\mathcal{Q}$. Even if the correction changes other property frequencies, its lower total loss contradicts optimality. Thus all m equalities hold at the same minimizer.

Example: matching tags. Consider four-token documents: an opening tag, two body words, and a closing tag. Treat each tag as one token. The source distribution p chooses `<note>` or `<quote>` as the opening tag, each with probability 1/2. Independently, it chooses the body `cats purr` or `dogs bark`, each with probability 1/2, and adds the matching closing tag. Thus p assigns probability 1/4 to each of four documents and zero to all others.

For each $\rho \in [0, 1]$, define a model q_ρ by the following generation rule:

1. Generate `<note>` or `<quote>`, each with probability 1/2.
2. At each of the two body positions, independently choose `cats`, `purr`, `dogs`, or `bark`, each with probability 1/4.
3. Generate the matching closing tag with probability $1 - \rho$ and the other closing tag with probability ρ .

The learner chooses from the model class

$$\mathcal{Q} = \{q_\rho : \rho \in [0, 1]\}.$$

All models use the same opening-tag and body-word probabilities; choosing a model amounts to choosing its mismatch probability ρ . The source couples its two body words, whereas these models generate them independently, so $p \notin \mathcal{Q}$.

Let B be the event that the opening and closing tags do not match. Under q_ρ , the closing tag mismatches with probability ρ after every opening and body, so $q_\rho(B) = \rho$. The source always matches its tags, so $p(B) = 0$.

The correction stays in the class. Fix an opening tag and body. Under q_ρ , the matching and mismatching closing tags have conditional probabilities $1 - \rho$ and ρ . The correction multiplies the mismatching weight by $e^{-\eta}$. Dividing the two new weights by their sum gives the corrected mismatch probability

$$\rho' = \frac{\rho e^{-\eta}}{1 - \rho + \rho e^{-\eta}}. \quad (10)$$

The sum $1 - \rho + \rho e^{-\eta}$ is the same for every opening and body. Reweighting therefore multiplies the total probability of every such prefix by the same factor. Normalizing the document distribution cancels this factor, preserving all opening and body probabilities. Only the closing prediction changes. Thus $q_\rho^{(\eta)} = q_{\rho'}$, and since $\rho' \in [0, 1]$ for every $\eta \in \mathbb{R}$, the correction stays in $\mathcal{Q}$.

Since $p(B) = 0$, the preceding optimality argument forces any population minimizer with finite loss to have $\rho = 0$. Thus $q^* = q_0$, and the best model always matches the tags.

KL can still be large. The model q_0 assigns each source document probability $(1/2)(1/4)^2 = 1/32$: the opening tag has probability $1/2$, each body word has probability $1/4$, and the closing tag is correct with probability one. The source assigns each of these documents probability $1/4$. Hence

$$\text{KL}(p||q^*) = \log \frac{1/4}{1/32} = \log 8, \quad q^*(B) = p(B) = 0.$$

For example, it generates `<note> cats bark </note>` with positive probability, although the source never does. The model matches the tags while failing to capture which words go together.

4 Misspecification and Generation Error

The preceding argument used the freedom to change the mismatch parameter ρ . A class that excludes the document correction may still contain a model with a small error on the property. How does population loss choose among the available models in such a class?

When $p \notin \mathcal{Q}$, we say the model class is *misspecified*. Minimizing population loss selects

$$q^* \in \arg \min_{q \in \mathcal{Q}} L_{\text{seq}}(q) = \arg \min_{q \in \mathcal{Q}} \text{KL}(p||q).$$

Common and rare documents. We use a two-token simplification of a construction of Rohatgi, Block, Huang, Krishnamurthy, and Foster [4, Propositions D.7–D.8]. Use the same tag format with no body, so each document consists of an opening tag and a closing tag. The source distribution p opens with `<note>` with probability $3/4$ and `<quote>` with probability $1/4$, and always uses the matching closing tag.

The learner’s model class is $\mathcal{Q} = \{q_{\text{note}}, q_{\text{quote}}\}$. Training chooses between these two fixed distributions. Both predictors use the correct opening-tag probabilities. They differ only in their predictions of the closing tag:

- q_{note} always closes a **note** correctly. It closes a **quote** correctly with probability $1/16$, and uses `</note>` otherwise.
- q_{quote} always closes a **quote** correctly. After a **note** opening, it chooses either closing tag with probability $1/2$.

The first model gets common notes right but assigns very little probability to the correct closing tag on a rare quote. The second gets quotes right but makes errors on common notes. The class contains only these two models, so the learner cannot combine their correct predictions. Neither model always matches the tags, so $p \notin \mathcal{Q}$.

Keep B as the event that the tags do not match. We have $p(B) = 0$, and $q(B)$ is the model’s generation error. Multiplying each opening probability by the corresponding closing probability gives Table 1.

Document or event	p	q_{note}	q_{quote}
<code><note> </note></code>	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{8}$
<code><quote> </quote></code>	$\frac{1}{4}$	$\frac{1}{64}$	$\frac{1}{4}$
B : mismatched tags	0	$\frac{15}{64}$	$\frac{3}{8}$

Table 1: The two models make errors on different document types: q_{note} makes errors only on rare quotes, while q_{quote} makes errors only on common notes.

4.1 The Correction Leaves the Class

Consider correcting q_{quote} for the event B . All of its mismatches begin with `<note>`. Downweighting these documents removes some probability from notes while leaving all quote probabilities unchanged before normalization. Normalizing therefore increases the probability of a quote opening above $1/4$. Both models in the class fix this probability at $1/4$, so the corrected distribution is outside $\mathcal{Q}$.

Proposition 3.1 gives a loss-decreasing correction, but the learner must choose between the two original models.

4.2 What Does Population Loss Select?

Generation error counts how often a sampled document is wrong. Log loss also depends on how little probability the model assigns to a correct source document. For a fixed opening tag, assigning probability δ to the correct closing tag gives mismatch probability $1 - \delta$ and log loss $\log(1/\delta)$. As δ approaches zero, the error approaches one while the loss grows without bound. A very small probability on a rare correct output can therefore outweigh more frequent errors elsewhere.

Generation error. The model q_{note} can only go wrong on quotes, which make up one quarter of its outputs. Its error is therefore at most $1/4$. The model q_{quote} goes wrong on half of its notes, giving error $(3/4)(1/2) = 3/8$. Thus q_{note} generates fewer mismatches.

Population loss. Only the closing predictions contribute to excess loss, since both models fit the opening-tag distribution. Under q_{note} , the penalty is $\log 16$ on a quote and zero on a note. Under q_{quote} , it is $\log 2$ on a note and zero on a quote. Averaging over the source frequencies gives

$$\begin{aligned}\text{KL}(p\|q_{\text{note}}) &= \frac{1}{4} \log 16 = \log 2, \\ \text{KL}(p\|q_{\text{quote}}) &= \frac{3}{4} \log 2.\end{aligned}$$

The population-loss minimizer is therefore $q^* = q_{\text{quote}}$, the worse of the two models for the mismatch property.

Quotes occur one third as often as notes, but the first model’s excess loss on a quote is four times the second model’s excess loss on a note. Avoiding the large penalty on rare quotes wins the loss comparison, even though this increases the frequency of mismatched tags.

The same comparison holds for TV. Both models assign at most the source probability to each correct document, so their only excess mass is on mismatches. Hence $\text{TV}(p, q) = q(B)$ for both.

How large can the gap be? Let quotes have source probability $\varepsilon \in (0, 1/4]$, and let q_{note} assign probability $\delta \in (0, 1]$ to the correct closing tag on a quote. Both models still fit the opening-tag distribution. The contribution from quotes is

$$q_{\text{note}}(B) = \varepsilon(1 - \delta) \leq \varepsilon, \quad \text{KL}(p\|q_{\text{note}}) = \varepsilon \log(1/\delta).$$

The generation error is bounded by the frequency of quotes, however small δ becomes. The log loss has no such bound. Choosing $\delta = 2^{-1/\varepsilon}$ keeps the excess loss at $\log 2$, even as quotes become rarer. This recovers the example above when $\varepsilon = 1/4$.

Keep q_{quote} ’s probability of closing a note correctly at $1/2$. Its error is $(1 - \varepsilon)/2$, and its excess loss is $(1 - \varepsilon) \log 2 < \log 2$, so population loss still selects it. As ε tends to zero, the first model’s generation error tends to zero, while the model selected by population loss has error tending to $1/2$. Its error can therefore be an arbitrarily large multiple of the smallest error in the class. The same separation holds for TV, since it equals the mismatch probability for both models. Figure 2 shows this dependence on the frequency of quotes.

The construction uses a very small probability for one correct closing token. Rohatgi et al. also obtain such a separation by accumulating log-loss penalties over a long continuation, while bounding the ratio of source to model probabilities at each prediction [4, Proposition D.7]. Both KL divergences tend to $\log 2$, so Pinsker’s inequality gives a constant bound on TV.

Further question: computing the correction. Our closure assumption concerns distributions over complete documents. What computation does a next-token predictor need to implement the correction? Cao and Vempala [3, Section 1 and Appendix B] illustrate the computational difficulty with prime factorization. Define a source distribution p by choosing two independent uniformly random b -bit primes r, s , computing $N = rs$, and printing

The prime factors of N are r and s .

Treat each digit as a token. Generating a document this way takes expected polynomial time in b . Checking a completed document is also efficient: test that the reported factors are prime and that their product is N .

Now prompt a predictor with “The prime factors of N are” and let it generate the rest. The source chose the factors before writing N , while the predictor must recover them from the supplied

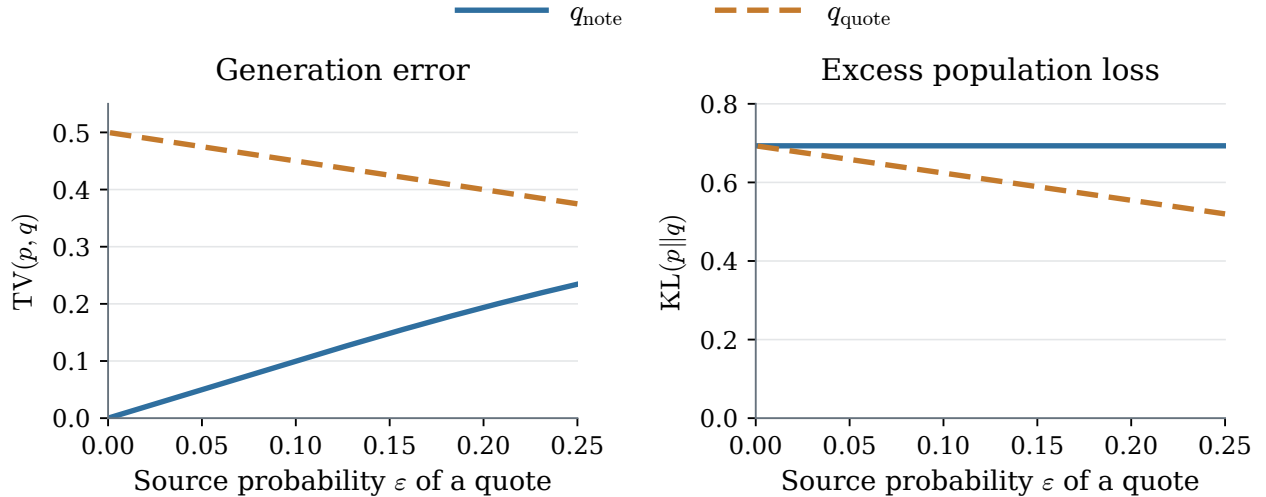


Figure 2: As quotes become rarer ($\varepsilon \rightarrow 0$), q_{note} approaches the source in TV. Population loss prefers q_{quote} for every $\varepsilon > 0$, although its generation error approaches $1/2$.

product. An efficient predictor that reliably completes these prompts would give an efficient algorithm for factoring products of random primes, contrary to the usual classical hardness assumption.

Let B be the event that the reported factorization is incorrect. The source has $p(B) = 0$, and an efficient test detects every such error. The correction in (8) downweights incorrect documents. To implement the new prediction for the first digit of a factor, we must account for the valid completions beginning with each possible digit. Testing a completed answer does not directly compute these probabilities.

Cao and Vempala [3, Lemmas 1–3] construct corrected neural predictors for tests of the next k tokens after random source prefixes. They quantify the loss improvement and the additional model size and computation. Their running-time bounds allow exponential dependence on k , consistent with the difficulty of completing factorization prompts.

References

- [1] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*, second edition. Wiley, 2006.
- [2] David Alvarez-Melis, Vikas Garg, and Adam Kalai. *Are GANs overkill for NLP?* NeurIPS, 2022. [Paper](#).
- [3] Xinyuan Cao and Santosh S. Vempala. *Provable Long-Range Benefits of Next-Token Prediction*. STOC, 2026. [arXiv:2512.07818](#).
- [4] Dhruv Rohatgi, Adam Block, Audrey Huang, Akshay Krishnamurthy, and Dylan J. Foster. *Computational-Statistical Tradeoffs at the Next-Token Prediction Barrier: Autoregressive and Imitation Learning under Misspecification*. COLT, 2025, extended abstract in PMLR 291:4831–4837. [Proceedings](#); [full paper](#).